

Docx 文件读取解析

docx 文件组织形式

文件组织形式与核心 xml 文件

docx 文件本质是一个 **ZIP 格式的压缩包**，解压后会生成一个包含多个子文件夹和 XML 文件的目录。例如：

```
unzip example.docx -d example_folder
```

解压后的文件**会存储在用户指定的目录**（如 example_folder）中，典型目录结构如下：

```
example_folder/
├── [Content_Types].xml
├── _rels/
├── docProps/
│   ├── app.xml
│   └── core.xml
└── word/
    ├── document.xml#文档主体内容（文字/图片/表格）
    ├── styles.xml
    └── numbering.xml
├── media/ # 存放图片/表格文件
└── _rels/ # 关系定义，映射图片 id 和路径关系
    ├── document.xml.rels # 正文与其他部分的关系
    └── ... # 其他关系文件
```

可以看到,word 解压之后会得到一个文件夹,里面存放着使得 word 能够展现到用户眼前的基础文件和支持文件,这里有 3 个值得我们注意的文件:

document.xml:这个文件是 word 的主体内容,存放着 word 的文字,表格的具体内容和图片的指代符号 rid;也存放着文字,段落的属性比如字体大小,颜色,段落间距等信息;这些内容以命名空间的形式分布(类似于括号表示范围).

media/:这个文件夹主要存放图片;

document.xml.rels:这个文件主要用来表示 document.xml 文件中的图片 ID 和 media 下的图片的对应关系,经常在 **Python 获取 word 里的图片**时使用。

每个 .rels 文件包含多个 **<Relationship>** 条目,格式如下:

```
<Relationships
xmlns="http://schemas.openxmlformats.org/package/2006/relationships">
<Relationship
Id="rId1" <!-- 关系唯一标识符 -->
Type="RelType" <!-- 关系类型 -->
Target="Target" <!-- 目标文件路径 -->
TargetMode="External/Internal" <!-- 目标位置 -->
```

</Relationships>

RelType（关系类型） 常见类型及含义：

RelType 值	含义
http://schemas.openxmlformats.org/officeDocument/2006/relationships/body	正文内容
http://schemas.openxmlformats.org/officeDocument/2006/relationships/styles	样式定义
http://schemas.openxmlformats.org/officeDocument/2006/relationships/image	图片资源
http://schemas.openxmlformats.org/officeDocument/2006/relationships/hyperlink	超链接

Target（目标路径）

相对路径：相对于当前目录的路径（如 media/image1.png）。

外部路径：用于链接外部资源（如 http://example.com/file.pdf）。

下面为全部的 xml 文件（了解即可）：

XML 文件	功能说明
document.xml	文档主体内容（文字/图片/表格）
styles.xml	段落/字符样式定义（如标题格式、字体颜色）
numbering.xml	自动编号和项目符号规则
footnotes.xml	脚注内容
endnotes.xml	尾注内容
settings.xml	文档全局设置（如默认视图模式）
theme/theme1.xml	颜色/字体主题配置

xml 命名空间与符号解析

文档的组件元素如段落，表格，文字，图片等，在 xml 文件里需要使用前后 2 个符号包裹起来（像括号一样），来表示包裹起来的内容的**类别**，比如对于 word 里的文字，“测试文本”，体现在 xml 文件里的基本单元是 <w:t>测试文本</w:t>，这里的“t”是 text 的首字母，那么“w”是什么呢？这个就要讲到 xml 文件的命名空间。

为什么需要命名空间呢？因为 xml **类别名称有限**，存在同名但不同内容的情况，就比如一个人，身份证号是唯一的自然就不需要额外的前缀，但是人的姓名是有可能相同的，这个时候就需要添加前缀，比如某某省市区公司学校，这里的前缀就是 xml 的命名空间。XML 命名空间（Namespace）通过**唯一标识的 URI**（统一资源标识符）来区分同名元素的语义，确保不同来源的元素在同一个文档中共存而不冲突。

下面举个实例说明一下是如何冲突的：
假设一个文档需要同时包含：

1. 图书信息（来自图书管理系统）
2. 订单中的商品项（来自电商系统）

两者都使用<book>元素，但语义不同：

图书信息中的<book>包含书名、作者、ISBN 等。

订单中的<book>包含商品 ID、数量、价格等。

在 xml 文件里这样表示：

```
<document>
  <book>
    <title>XML 权威指南</title>
    <author>John Doe</author>
  </book>
  <book>
    <item_id>12345</item_id>
    <quantity>2</quantity>
  </book>
</document>
```

问题：两个<book>元素同名但语义不同，在获取信息时无法区分这二者，比如想要获取图书书名作者这些信息的时候，订单商品 ID 这些信息也会加载出来。

那么加了命名空间后是如何实现区分的呢？下面介绍一下：

第一步，通过 **xmlns 属性** 声明命名空间，

```
<document
  xmlns:book="http://example.com/books"
  xmlns:order="http://example.com/orders">
  相当于以字典的形式将"http://example.com/books"和
"http://example.com/orders">这 2 个 URI 分别赋值给了 xmlns:book 和 xmlns:order.
```

第二步，在 xml 元素前添加命名空间来进行区分：

```
<!-- 图书信息 -->
<book:book>
  <book:title>XML 权威指南</book:title>
  <book:author>John Doe</book:author>
</book:book>

<!-- 订单中的书籍商品 -->
<order:book>
  <order:item_id>12345</order:item_id>
  <order:quantity>2</order:quantity>
</order:book></document>
```

这样在实际读取的时候，

- 通过前缀（如 book:和 order:）区分不同命名空间的元素。
- 命名空间 URI（如 http://example.com/books）确保全局唯一性。

基本组件

Word 文档虽然以直观的形式呈现到眼前，但是其具体的内容组织形式，文档格式等属性也是在 xml 文件内规定的。概括说来，docx 文件整体为 **document（整个文档）**，下面又细分为**段落（paragraph）**，表格（table）2 个大类，段落和表格内有文本和图片（包括行内图片和独立图片）。这些基本组件又细分为不同的属性，比如段落有间距，文字有不同的格式（加粗颜色等）。

在 Python 语言中，文档，段落，表格都是以对象的数据类型存在，可以通过类方法和属性直接获取到。

document.xml 的命名空间

首先看一下 word 解析后的 document.xml 文件里的常用命名空间，一般包括下面几种映射关系：

- “w:”：段落、文本（w:p、w:t）
- “wp:”：图片位置（wp:inline）
- “a:”：图片数据（a:blip）
- “r”：关系

如下图

```
<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<w:document
  xmlns:w="http://schemas.openxmlformats.org/wordprocessingml/2006/main"
  xmlns:r="http://schemas.openxmlformats.org/officeDocument/2006/relationships"
  xmlns:wp="http://schemas.openxmlformats.org/drawingml/2006/wordprocessingDrawing"
>
  <w:body>...</w:body>
</w:document>
```

结合前面所讲的 word 文档的组件，xml 文件里的基本组件主要也是下面几种：

第一层级<document>

第二层级：段落<w:p>,表格<w:tbl>;

第三层级：文本，<w:t>，图片位置<wp:inline>，图片数据<a:blip>。

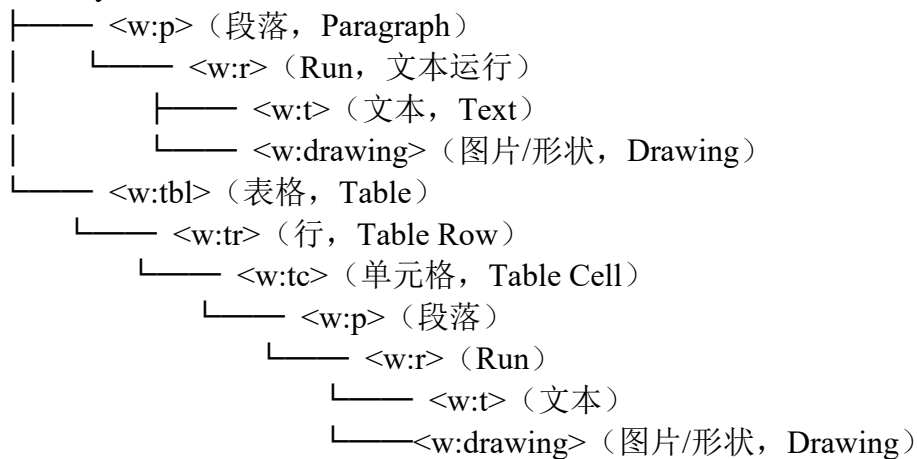
还有比较特殊的容器，一种是属性，表示段落和文字的格式，<w:rPr>等，表示在该容器范围里的组件元素的格式是统一的；

另一个重要的元素是<run>，存储每一个不同属性的文本，图片等更精细组件的容器。这个在解析底层 xml 文件获取精细获取文字，图片等相对位置时很有用。下面看一些实例是怎么样的：

运行块（Run）

在 Open XML 中，文档内容的层级结构是：

<w:body>（文档主体）



其中：

- **<w:r> (Run)**：核心作用是“格式容器”——表示一组格式完全一致的内容（文本、图片等）。
- **文本**：存储在 <w:t> 标签中，是 Run 的“内容”之一。
- **图片**：内嵌型图片存储在 <w:drawing> 中，通常也作为 Run 的“内容”。
- **表格**：独立的结构化元素（<w:tbl>），与段落 <w:p> 同级，但表格内部的单元格内容仍需通过“段落 → Run → 文本 / 图片”来承载。

段落 (Paragraph) 与文本

```
<w:p>
  <w:pPr>                                <!-- 段落属性 -->
    <w:jc w:val="center"/>              <!-- 居中对齐 -->
    <w:spacing w:before="200"/>          <!-- 段前间距 -->
  </w:pPr>
  <w:r>                                    <!-- 文本运行块 (Run) -->
    <w:t>Hello World</w:t>              <!-- 文本内容 -->
  </w:r>
</w:p>
```

<w:p>：段落容器。

<w:pPr>：段落属性（对齐、缩进、间距等）。

<w:jc w:val="center">：对齐方式（left/center/right）。

<w:spacing w:before="200">：段前间距（单位：twip，1 twip=1/20 磅）。

文本是 Run 最常见的内容，文本必须放在 **Run** 里，没有 Run 就无法直接存储文本。

<w:p> <!-- 段落 -->

<w:r> <!-- 第 1 个 Run：格式为“加粗” -->

<w:rPr> <!-- Run 的格式定义 (Run Properties) -->

<w:b/> <!-- 加粗 -->

```

</w:rPr>
<w:t>这是加粗文本</w:t> <!-- 文本内容 -->
</w:r>

<w:r> <!-- 第 2 个 Run: 格式为“红色、斜体” -->
  <w:rPr>
    <w:i/> <!-- 斜体 -->
    <w:color w:val="FF0000"/> <!-- 红色 -->
  </w:rPr>
  <w:t>这是红色斜体文本</w:t>
</w:r></w:p>

```

- 一个段落可以有**多个 Run**—— 因为不同格式的文本需要用不同的 Run 包裹（比如上例中“加粗”和“红色斜体”是两个 Run）。
- Run 的格式定义在 <w:rPr>（Run Properties）中，<w:t> 只存纯文本。

表格（Table）

表格在 word 文档属于第二层级，里面具体的文本，图片等内容是按照表格 table——行 row——单元格 cell 这个顺序获取的。

表格本身是与段落**同级**的独立元素（<w:tbl>），表格的结构（行、列、单元格）由 <w:tbl>/<w:tr>/<w:tc> 定义，与 Run 无关；

但表格的**单元格内容**（文字、图片）仍需通过“段落 → Run → 文本 / 图片”来承载。没有 Run，单元格里就无法显示内容。

```

<w:tbl> <!-- 表格容器（与段落同级） -->
  <w:tr> <!-- 表格行 -->
    <w:tc> <!-- 表格单元格，包含段落或嵌套内容 -->
      <w:p> <!-- 单元格里的段落 -->
        <w:r> <!-- 单元格里的 Run -->
          <w:t>单元格文本</w:t> <!-- 单元格里的文本 -->
        </w:r>
      </w:p>
    </w:tc>
  </w:tr>
</w:tbl>

```

图片（Drawing）与对象（**图片与文字的位置关系到底是什么样的，有没有绝对位置关系还是浮动的**）

1. 内嵌型**图片**

内嵌型图片（随文字流动的图片）必须放在 **Run** 里，用 `<w:drawing>` 标签包裹，作为 **Run** 的内容之一。

```
<w:p>
  <w:r> <!-- 包含图片的 Run -->
    <w:drawing> <!-- 图片容器 -->
      <wp:inline> <!-- 内嵌型图片（区别于浮动型） -->
        <a:graphic>
          <a:blip r:embed="rId5"/> <!-- 图片的引用 ID（对应 media 文件夹里的图片） -->
        </a:graphic>
      </wp:inline>
    </w:drawing>
  </w:r></w:p>
```

- 图片本身不直接存在于 XML 中，XML 里只存图片的引用 ID（`r:embed="rId5"`），实际图片文件在 .docx 解压后的 `word/media/` 文件夹里。
- 一个 **Run** 可以同时包含文本和图片（比如“文字 + 图片 + 文字”），但通常为了格式清晰，会把图片单独放在一个 **Run** 里。

2. 浮动图片

```
<w:p>
  <w:r>
    <w:drawing>
      <wp:anchor> <!-- 浮动图片 -->
        <wp:positionH relativeFrom="page"/>
        <a:graphic>...</a:graphic>
      </wp:anchor>
    </w:drawing>
  </w:r>
</w:p>
```

<wp:anchor>：浮动图片（可拖拽位置）。

换行符与超链接

前面已经介绍了运行块是如何包裹文字以及图片的，下面再介绍换行符和超链接。

1. 文本与换行符


```

<w:p>
  <w:r>
    <w:t>Line1</w:t>
  </w:r>
  <w:r>
    <w:br />                                <!-- 换行符 -->
  </w:r>
  <w:r>
    <w:t>Line2</w:t>
  </w:r>
</w:p>

```

<w:br>：强制换行符。

2. 超链接

```

<w:p>
  <w:hyperlink r:id="rId3">                <!-- 指向关系文件中的链接 -->
    <w:r>
      <w:rPr>
        <w:u w:val="single" />            <!-- 下划线 -->
      </w:rPr>
      <w:t>Click Here</w:t>
    </w:r>
  </w:hyperlink>
</w:p>

```

<w:hyperlink>：超链接容器，通过 r:id 关联到 word/_rels/document.xml.rels 中的 URL。

组件元素总结

符号/标签	含义
<w:p>	段落容器
<w:r>	文本运行块（同一段内不同格式的文本块）
<w:t>	纯文本内容
<w:pPr> / <w:rPr>	段落/文本属性定义（对齐、字体、颜色等）
<w:tbl>	表格容器
<w:drawing>	图片/图形容器
r:id	资源引用（指向 document.xml.rels 中的图片、超链接等）
w:rsidR	修订标识符（用于跟踪文档修改）

基于 python-docx 解析

python-docx 的设计逻辑是：用 **Python** 类映射 .docx 的 **XML** 节点，每个类对应文档中的一个“构件”（如段落、表格），类的属性 / 方法对应构件的“内容”或“操作”。

高级 api 的读取和 docx 文档的树状层级组织形式是一致的，即先从最上层的文档 **document**，文档下层有段落 **paragraph** 和表格 **tables**，段落又有文字，表格有行，行又有单元格。

```
from docx import Document
```

```
doc=Document(self.filepath)#将文件路径作为 Document 的参数读取到 doc 对象，这个对象代表整个文档；
```

```
for paragraph in doc.paragraphs:#遍历段落
```

```
    print(paragraph.text)#读取段落文字
```

```
for table in doc.tables:#遍历表格
```

```
    for row in table.rows:#遍历单个表格的行
```

```
        for cell in row.cells:#便利每个表格每个行的单元格
```

```
            print(cell.text)#cell.text 获取单元格里的文字
```

标红 **print** 处为真正提取内容文字的部分，从上面的代码以及输出可以看到：

（1）这种方法会把段落和表格区分开来，所有的段落提取内容，所有的表格提取内容，段落和表格的相对位置关系丢失；

（2）遍历段落合集，每个段落有文字属性，可以读取到纯文字；表格遍历是先遍历行，然后在行上遍历单元格，单元格有 **text** 文字属性，可以提取单元格里的文字。

（3）段落 **paragraph** 下没有图片属性，无法获取图片。

（4）这里使用 **for** 循环遍历 **paragraphs** 和 **tables**，还有没有其它迭代遍历的方式（**iterchildren?**）？

1. 顶层容器：Document 类

Document 是整个 .docx 文件的映射，是所有操作的入口。

维度	说明
数据类型	docx.document.Document（Python 类） - paragraphs : 文档中所有段落（Paragraph 对象列表） - tables : 文档中所有表格（Table 对象列表）
核心属性	- inlineshapes : 文档中内嵌图片 / 形状（InlineShape 对象列表） - styles : 文档样式库（Styles 对象） - core_properties : 文档元数据（作者、标题等）
核心方法	- add_paragraph(text="", style=) : 添加段落

维度	说明
	- add_table(rows, cols, style=): 添加表格
	- add_picture(image_path, width, height=): 添加图片
	- save(path): 保存文档

关键操作示例:

```
from docx import Document
# 1. 打开文档 (创建 Document 实例)
doc = Document("example.docx")
# 2. 遍历所有段落 for para in doc.paragraphs:
    print(para.text) # 输出段落文本
# 3. 保存修改后的文档
doc.save("modified.docx")
```

2. Paragraph 类 (段落) 与 Run 类 (文本块)

.docx 中文字的核心逻辑是: **段落 (Paragraph)** 包含多个 **文本块 (Run)**。

- **Paragraph**: 代表一个完整段落 (对应 XML 中的 <w:p> 节点), 负责段落级格式 (对齐、缩进)。
- **Run**: 代表段落中**格式连续**的一段文字 (对应 XML 中的 <w:r> 节点), 负责字符级格式 (加粗、字体)。

(1) Paragraph 类

维度	说明
数据类型	docx.text.paragraph.Paragraph (Python 类)
	- text: 段落的纯文本 (自动合并所有 Run 的文本)
	- runs: 段落内的 Run 对象列表
核心属性	- style: 段落样式 (ParagraphStyle 对象)
	- alignment: 段落对齐方式 (枚举, 如 LEFT/CENTER)
	- paragraph_format: 段落格式 (ParagraphFormat 对象, 含缩进、行间距等)
	- add_run(text="", style=None): 在段落末尾添加 Run
核心方法	- clear(): 清空段落内容 (保留格式)
	- insert_paragraph_before(text="", style=None): 在当前段落前插入新段落

(2) Run 类

维度	说明
数据类型	docx.text.run.Run (Python 类)
核心属性	- text: 当前 Run 的文本内容

维度	说明
	- bold/italic/underline: 布尔值, 控制加粗 / 斜体 / 下划线 - font: 字体对象 (Font 类, 含 name (字体名)、size (字号)、color (颜色) 等属性) - style: 字符样式 (CharacterStyle 对象) - add_text(text): 在 Run 末尾追加文本
核心方法	- add_break(type=WD_BREAK.LINE): 添加换行符 / 分页符 - add_picture(image_path, width=None, height=None): 在 Run 中嵌入图片

表格元素: Table/Row/Cell 类

.docx 中表格的核心逻辑是: Table (表格) 包含 Row (行), Row 包含 Cell (单元格)。

(1) Table 类

维度	说明
数据类型	docx.table.Table (Python 类)
	- rows: 表格的行列表 (Row 对象列表)
核心属性	- columns: 表格的列列表 (Column 对象列表) - style: 表格样式 (TableStyle 对象, 如 “网格型”) - autofit: 布尔值, 是否自动调整列宽
核心方法	- add_row(): 在表格末尾添加一行 - add_column(width): 添加一列 (需指定宽度) - cell(row_idx, col_idx): 获取指定位置的单元格 (Cell 对象) - row_cells(row_idx): 获取某一行的所有单元格

(2) Row 类

维度	说明
数据类型	docx.table.Row (Python 类)
	- cells: 当前行的单元格列表 (Cell 对象列表)
核心属性	- height: 行高 (Length 对象, 如 Pt(20)) - height_rule: 行高规则 (WD_ROW_HEIGHT_RULE 枚举, 如 AT_LEAST/EXACTLY)
核心方法	- cell(col_idx): 获取当前行第 col_idx 列的单元格

(3) Cell 类

维度	说明
数据类型	docx.table.Cell (Python 类)

维度	说明
核心属性	- text: 单元格的纯文本（自动合并内部段落的文本）
	- paragraphs: 单元格内的段落列表（Paragraph 对象列表，单元格可包含多段落）
	- tables: 单元格内嵌套的表格列表（Table 对象列表）
	- vertical_alignment: 垂直对齐方式（WD_ALIGN_VERTICAL 枚举）
	- width: 单元格宽度
核心方法	- add_paragraph(text="", style=None): 在单元格内添加段落
	- add_table(rows, cols): 在单元格内嵌套表格
	- merge(other_cell): 与另一个单元格合并

图片元素：InlineShape 类

.docx 中图片分为“内嵌型”（InlineShape，随文字流动）和“浮动型”（Shape，独立定位），python-docx 主要支持 InlineShape。

维度	说明
数据类型	docx.shape.InlineShape（Python 类）
核心属性	- type: 形状类型（WD_INLINE_SHAPE_TYPE 枚举，如 PICTURE（图片）、CHART（图表））
	- width/height: 图片宽度 / 高度（Length 对象）
	- alternative_text: 图片替代文本（Alt Text）
核心方法	- picture: 图片对象（Picture 类，可获取图片原始数据）
	- crop_top/crop_bottom/crop_left/crop_right: 裁剪图片（设置裁剪比例）
关键操作示例：	

```
from docx import Document
from docx.shared import Inches
```

```
doc = Document()
# 1. 添加图片（指定宽度，高度自动等比缩放）
doc.add_picture("image.png", width=Inches(3))
# 2. 读取并保存现有文档的图片
doc = Document("example.docx")
for idx, shape in enumerate(doc.inlineshapes):
    if shape.type == 1: # 1 代表图片（WD_INLINE_SHAPE_TYPE.PICTURE）
        # 获取图片原始数据（blob）
        image_blob = shape.image.blob
        # 保存到本地
        with open(f'extracted_image_{idx}.png', "wb") as f:
            f.write(image_blob)
```

那么图片该如何遍历呢？分两步，第一步是找到图片，第二步是利用 ocr 图片检测工具识别图片上的文字。（如何保证图片和文字段落的相对位置不变？）那么第一步该如何找到图片呢？两种方法，一种是借助文档的关系类型来判断 rel.reltypes，然后将图片另存到一个路径下 `for rel in doc.part.rels.values():` 这种方

法不能保证图片和文字的相对位置顺序，不建议使用。第二种读取图片的方法是
基于 XPath 和 run 的方法

代码实例

```
from docx import Document
from docx.document import Document as DocxDocument
from docx.oxml.shape import CT_Picture
from docx.oxml.table import CT_Tbl
import pytesseract
from PIL import Image

def process_docx_with_runs(docx_path):
    doc = Document(docx_path)
    output = []
    # 遍历所有段落
    for para in doc.paragraphs:
        para_text = para.text.strip()
        if para_text:
            output.append(('text', para_text))
    # 遍历段落内的 Run
    for run in para.runs:
        # 检测 Run 中的图片
        for elem in run._element.xpath('.//pic:pic', namespaces={'pic':
http://schemas.openxmlformats.org/drawingml/2006/picture}):
            image_part = None
            # 获取图片资源 ID
            blip = elem.xpath('.//a:blip', namespaces={'a':
http://schemas.openxmlformats.org/drawingml/2006/main'}))[0]
            image_id =
            blip.get('{http://schemas.openxmlformats.org/officeDocument/2006/relationships}embed')
            # 通过文档的 Relationships 获取图片路径
            image_part = doc.part.related_parts[image_id]
            # 保存图片并 OCR
            with open('temp_image.png', 'wb') as f:
                f.write(image_part.blob)
            ocr_text =
            pytesseract.image_to_string(Image.open('temp_image.png'))
            output.append(('image_ocr', ocr_text))
    # 处理表格
    for table in doc.tables:
        table_data = []
        for row in table.rows:
            row_data = [cell.text.strip() for cell in row.cells]
```

```

        table_data.append(' | '.join(row_data))
    output.append(('table', '\n'.join(table_data)))
return output

```

docx2txt: 纯文本字符串

数据类型: str (Python 原生字符串)。

特点: 直接返回文档的纯文本内容, 丢失所有格式、结构 (表格 / 图片)。

```

import docx2txt
text = docx2txt.process("example.docx") # text 是一个字符串
print(text[:100]) # 输出前 100 个字符

```

lxml 直接解析 XML: _Element 对象

数据类型: lxml.etree._Element (lxml 的 XML 节点对象)。

特点: 直接操作 .docx 解压后的 XML 文件, 最灵活但需理解 XML 结构。

.docx 本质是 ZIP 压缩包, 解析前需:

1. 用 zipfile 解压并读取内部 XML 文件;
2. 用 lxml 解析 XML (支持 XPath, 比 Python 原生 xml.etree 更强大)。

xpath 读取 xml 内容

了解了 xml 内容的基本结构, 接下来需要学习怎么获取到这些内容, 简单讲分 2 步, 第一步是知道想要获取的内容在哪里 (获取内容的位置), 第二步将内容提取出来。找到想要获取内容的位置依赖的就是 **xpath 语句**。

Xpath 语句可分为 3 部分, 第一部分是结合命名空间找到想要获取内容的绝对位置, 第二部分是通过 **表达式** 根据需求获得想要的内容元素, 第三部分是返回这些元素包裹的内容。

在 XPath 中引用命名空间元素需通过 **namespaces 参数** (如 Python 的 lxml 库), 首先看一下 2 种注册命名空间的方法:

在 XPath 中, 命名空间需要通过前缀映射到 URI, 或通过函数直接匹配元素。以下是两种常用方法: **两种的区别和适用范围是什么**

1. 注册命名空间前缀 (推荐)

通过 lxml 注册命名空间前缀:

```

from lxml import etree
# 定义命名空间映射
nsmap = { 'w': 'http://schemas.openxmlformats.org/wordprocessingml/2006/main',
'pic': 'http://schemas.openxmlformats.org/drawingml/2006/picture',

```

```
'a': 'http://schemas.openxmlformats.org/drawingml/2006/main',
'r': 'http://schemas.openxmlformats.org/officeDocument/2006/relationships' }
# 解析 XML
tree = etree.parse("document.xml")
root = tree.getroot()
这里的解析会将 document.xml 的内容按照树的结构返回根节点。
# 使用前缀直接编写 XPath
pic_elements = root.xpath('//pic:pic', namespaces=nsmap)
2. 手动匹配命名空间（兼容性更强）
通过 local-name() 和 namespace-uri() 函数动态匹配：
# 解析 XML
tree = etree.parse("document.xml")
root = tree.getroot()
# 查找所有 <pic:pic> 元素
pic_elements = root.xpath('//*[local-name()="pic" and
namespace-uri()="http://schemas.openxmlformats.org/drawingml/2006/picture"]')
```

XPath 核心符号

前面讲了如何选定想要查找元素的命名空间，接下来介绍一下在 xpath 语句中如何指定想要查找的元素。首先看一下 xpath 表达式中的符号的含义，其根本目的就是为了找到想要获取的元素的位置所在。这里值得注意的是 text() 这个表达式，其更像是一个函数用来获取元素下的文本。

符号/表达式	含义	示例
/	从根节点开始选择，或作为路径分隔符	/bookstore/book
//	全局搜索，递归查找所有层级的节点	//div 查找所有 <div> 元素
.	当前节点	./title 选择当前节点下的 <title>
..	父节点	../price 选择父节点下的 <price>
@	属性选择符	//img/@src 获取所有图片的 src 属性
*	通配符，匹配任意元素节点	//book/* 选择 <book> 的所有子元素
[]	谓语句（条件过滤）	//book[price>35] 选择价格大于 35 的书
text()	获取元素内的文本节点	//title/text() 获取标题文本
contains()	模糊匹配函数	//div[contains(@class, 'header')]
position()	节点位置索引	//book[position()=1] 选择第一本书

提取不同节点的 XPath 实例

1. 提取文本内容 (text())

```
//w:t/text()
```

含义：查找并返回所有 `<w:t>` 元素下的文本。

返回值类型：字符串列表（如 `['Hello World', 'Click Here']`）。

```
tree = etree.parse('document.xml')
```

```
text_nodes = tree.xpath('//w:t/text()', namespaces=namespaces)
```

```
print(text_nodes) # 输出: ['Hello World', 'Click Here']
```

2. 提取属性值 (@attribute)

```
//a:blip/@r:embed
```

含义：查找所有 `<a:blip>` 元素的 `r:embed` 属性值。

返回值类型：字符串列表（如 `['rId2']`）。

```
embed_ids = tree.xpath('//a:blip/@r:embed', namespaces=namespaces)
```

```
print(embed_ids) # 输出: ['rId2']
```

3. 提取特定元素 (<blip>)

```
//pic:blipFill/a:blip
```

含义：查找所有 `<pic:blipFill>` 下的 `<a:blip>` 元素。

返回值类型：元素节点列表（`lxml.etree.Element` 对象）。

```
blip_elements = tree.xpath('//pic:blipFill/a:blip', namespaces=namespaces)
```

```
for blip in blip_elements:
```

```
    print(blip.tag) # 输出: {http://schemas.../main}blip
```

```
    print(blip.get('{http://schemas.../relationships}embed')) # 输出: rId2
```

4. 条件过滤 ([] 和 contains())

```
//w:hyperlink[contains(@r:id, 'rId1')]
```

含义：查找 `r:id` 属性包含 `rId` 的所有 `<w:hyperlink>` 元素。

返回值类型：元素节点列表。

```
hyperlinks = tree.xpath("//w:hyperlink[contains(@r:id, 'rId1')]",  
namespaces=namespaces)
```

```
for link in hyperlinks:
```

```
    link_id = link.get('{http://schemas.../relationships}id')
```

```
    print(link_id) # 输出: rId1
```

核心流程与代码实例

以一个包含 段落、文本、表格、图片 的 `example.docx` 为例，逐步解析。

解压 .docx 并读取核心 XML

首先解压 `.docx`，读取两个关键文件：

- `word/document.xml`：文档主体内容（段落、表格、图片引用）；

- word/_rels/document.xml.rels: 图片等资源的关系映射（将 XML 中的 r:embed ID 映射到实际图片路径）。

```
import zipfile
from lxml import etree
def load_docx_xml(docx_path):
    """解压 docx 并返回核心 XML 根节点和关系映射"""
    with zipfile.ZipFile(docx_path, 'r') as z:
        # 1. 读取 document.xml (主体内容)
        document_xml = z.read('word/document.xml')
        # 2. 读取 document.xml.rels (资源关系映射)
        rels_xml = z.read('word/_rels/document.xml.rels')

        # 3. 解析 XML 并定义命名空间 (Open XML 必须)
        namespaces = {
            'w': 'http://schemas.openxmlformats.org/wordprocessingml/2006/main',
            'a': 'http://schemas.openxmlformats.org/drawingml/2006/main',
            'r':
'http://schemas.openxmlformats.org/officeDocument/2006/relationships'
        }

        # 4. 解析主体 XML 和关系 XML
        doc_root = etree.fromstring(document_xml)
        rels_root = etree.fromstring(rels_xml)

        # 5. 构建关系映射: {关系 ID: 资源路径}
        rels_map = {}
        for rel in rels_root.findall('.//r:Relationship', namespaces):
            rel_id = rel.get('Id')
            target = rel.get('Target') # 比如 'media/image1.png'
            rels_map[rel_id] = target

        return doc_root, rels_map, namespaces, z
# 加载示例文档
docx_path = 'example.docx'
doc_root, rels_map, namespaces, z = load_docx_xml(docx_path)
```

解析 段落 (Paragraph) 和 文本 (Text)

在 XML 中:

- 段落对应 <w:p> 标签;
- 段落内的 **格式单元 (Run)** 对应 <w:r>;
- 实际文本对应 <w:t> 标签 (需合并同一 <w:p> 下所有 <w:t> 的内容)。

```
def extract_paragraphs(doc_root, namespaces):
    """提取所有段落的纯文本"""
    paragraphs = []
    # 用 XPath 找到所有 <w:p> 节点（段落）
    for p in doc_root.findall('.//w:p', namespaces):
        # 提取当前段落内所有 <w:t> 节点的文本（合并 Run）
        text_parts = []
        for t in p.findall('.//w:t', namespaces):
            if t.text:
                text_parts.append(t.text)
        paragraph_text = ".join(text_parts)
        paragraphs.append(paragraph_text)
    return paragraphs
# 测试：提取并打印所有段落
paragraphs = extract_paragraphs(doc_root, namespaces)
print("=== 段落内容 ===")
for i, para in enumerate(paragraphs, 1):
    print(f"段落 {i}: {para}")
```

解析 表格（Table）

在 XML 中：

- 表格对应 <w:tbl> 标签；
- 表格行对应 <w:tr>；
- 单元格对应 <w:tc>；
- 单元格内的内容仍需通过 <w:p> → <w:r> → <w:t> 提取（复用段落解析逻辑）。

```
def extract_tables(doc_root, namespaces):
    """提取所有表格数据（返回列表：[表格 1, 表格 2, ...]，每个表格是行列表）
    """
    tables = []
    # 找到所有 <w:tbl> 节点（表格）
    for tbl in doc_root.findall('.//w:tbl', namespaces):
        table_data = []
        # 找到当前表格的所有 <w:tr> 节点（行）
        for tr in tbl.findall('.//w:tr', namespaces):
            row_data = []
            # 找到当前行的所有 <w:tc> 节点（单元格）
            for tc in tr.findall('.//w:tc', namespaces):
                # 单元格内的文本：复用段落提取逻辑
                cell_text_parts = []
                for p in tc.findall('.//w:p', namespaces):
                    for t in p.findall('.//w:t', namespaces):
```

```

        if t.text:
            cell_text_parts.append(t.text)
        cell_text = ".join(cell_text_parts)
        row_data.append(cell_text)
        table_data.append(row_data)
        tables.append(table_data)
    return tables
# 测试：提取并打印所有表格
tables = extract_tables(doc_root, namespaces)
print("\n=== 表格内容 ===")
for i, table in enumerate(tables, 1):
    print(f'表格 {i}:')
    for row in table:
        print(row)

```

解析 图片 (Image)

在 XML 中：

- 图片不直接存储在 document.xml 里，而是存 引用 ID (r:embed)；
- 需通过 rels_map 将引用 ID 映射为实际图片路径（如 word/media/image1.png）；
- 最后从 ZIP 包中提取图片二进制数据。

```

def extract_images(doc_root, rels_map, namespaces, z,
output_dir='extracted_images'):
    """提取所有图片并保存到本地"""
    import os
    os.makedirs(output_dir, exist_ok=True)

    image_count = 0
    # 找到所有 <a:blip> 节点（图片引用）
    for blip in doc_root.findall('.//a:blip', namespaces):
        # 获取图片的关系 ID (r:embed)
        rel_id = blip.get(f'{{{namespaces["r"]}}}embed')
        if not rel_id:
            continue

        # 通过关系 ID 找到图片在 ZIP 包中的路径
        image_path_in_zip = f'word/{rels_map[rel_id]}' # 比如
        'word/media/image1.png'

        # 从 ZIP 包中读取图片二进制数据
        try:
            image_data = z.read(image_path_in_zip)

```

```

        # 保存到本地
        image_ext = image_path_in_zip.split('.')[-1]
        output_path = os.path.join(output_dir,
fimage_{image_count}.{image_ext}')
        with open(output_path, 'wb') as f:
            f.write(image_data)
        print(f'已保存图片: {output_path}')
        image_count += 1
    except KeyError:
        print(f'未找到图片: {image_path_in_zip}')
# 测试: 提取图片
print("\n=== 提取图片 ===")
extract_images(doc_root, rels_map, namespaces, z)

```

关键组件解析逻辑总结

组件	XML 标签	解析核心逻辑
文档	<w:document>	解压 ZIP, 读取 word/document.xml, 定义命名空间。
段落	<w:p>	用 XPath //w:p 定位, 合并内部所有 <w:t> 文本。
文本	<w:t>	依附于 <w:r> (格式单元), 需合并同一 <w:p> 下的 <w:t> 以保留段落完整性。
表格	<w:tbl>	层级: <w:tbl> → <w:tr> → <w:tc> → <w:p> → <w:t>, 复用段落解析逻辑提取单元格文本。
图片	<a:blip>	1. 从 <a:blip> 获取 r:embed ID; 2. 用 rels_map 映射到 ZIP 内图片路径; 3. 提取二进制数据。